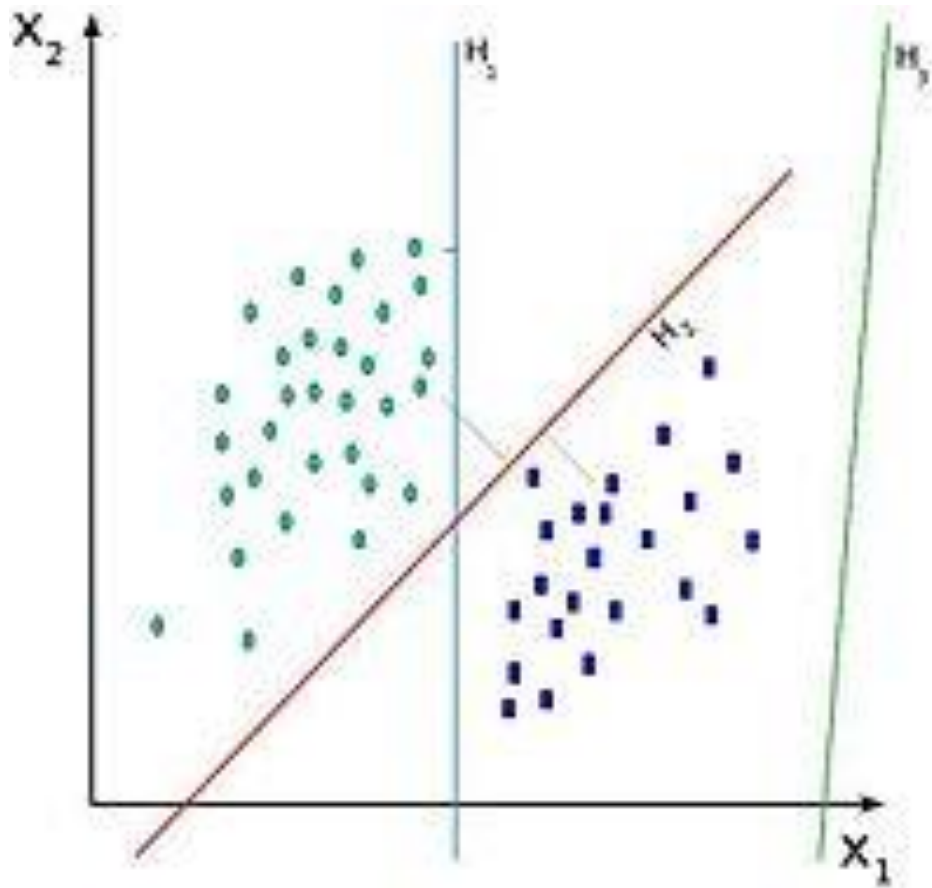


DIPLOMATURA EN CIENCIA DE DATOS

Maquinas de soporte vectorial

Introducción:

Vamos a imaginar que tenemos varios puntos de dos colores situados en un plano:



Tenemos a los puntos azules y a los puntos verdes. Nos proponemos encontrar una recta que sirva para separar los puntos verdes de los puntos azules.

En nuestro esquema la recta celeste y la recta roja serían candidatas mientras que la recta verde no lo consigue.

Fundamento:

Para poner el problema en términos más familiares para la ciencia de datos podemos decir que tenemos N casos (uno por cada punto) y cada caso cuenta con varios atributos. (dos por punto, su coordenada X y su coordenada Y) y lo que queremos encontrar es algún hiperplano (que es un conjunto con una dimensión menos que el espacio total en este caso sería $2 - 1 = 1$: una recta) que sirva para separar los casos según un resultado (en nuestro ejemplo el resultado es el color)

En general tendríamos:

N puntos de M atributos cada uno:

$$X_i \quad 1 \leq i \leq N$$

Cada X_i es un vector:

$$X_i = (X_{i1}, \dots, X_{iM})$$

Cada uno de los puntos presentaría un resultado categórico R_i

Es necesario, para poder aplicar esta técnica, que los X_i sean siempre numéricos para que tenga sentido representarlos en un espacio.

$$R_i \quad 1 \leq i \leq N$$

En principio está definida para dividir al conjunto de casos en dos categorías. Sin embargo hay dos estrategias que pueden permitirnos expandir a múltiples categorías:

- Subdividiendo sucesivamente cada categoría en nuevas.
- Haciendo $k(k-1)/2$ modelos siendo k la cantidad de categorías.

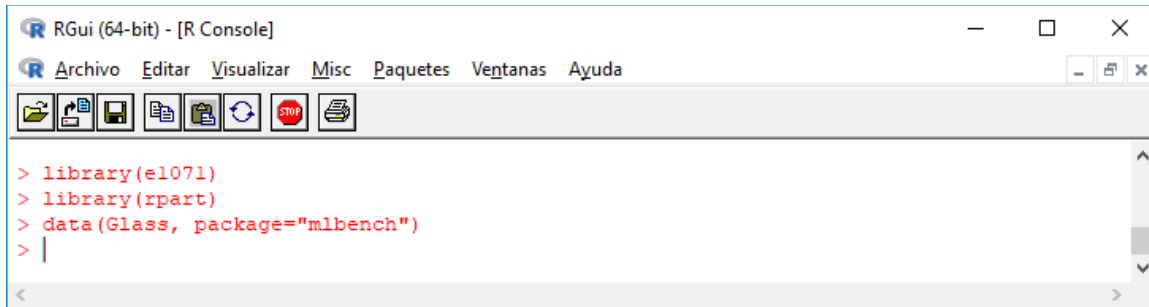
Debido a la complejidad matemática de esta técnica, no entraremos en detalles de su fundamentación y nos enfocaremos más en la implementación en R y en algunos ejemplos.

Creemos que, al menos para empezar, esta metodología es más adecuada, ya que nos permite ir introduciéndonos en un tema complejo poco a poco y a través de ejemplos. Alentamos al lector a usar esta técnica, empezando por algunos casos sencillos, y luego investigar más en profundidad los fundamentos en los que se basa.

Implementación en R:

El paquete *e1071* ofrece una interface para poder aplicar esta técnica y es la que vamos a usar para ejemplificar en esta sección. El paquete *rpart*, viejo conocido nuestro, nos servirá para generar un árbol de decisión sobre los mismos datos de manera de comparar el resultado.

El paquete *mlbench* tiene el dataset Glass que nos servirá para desarrollar el ejemplo.



```

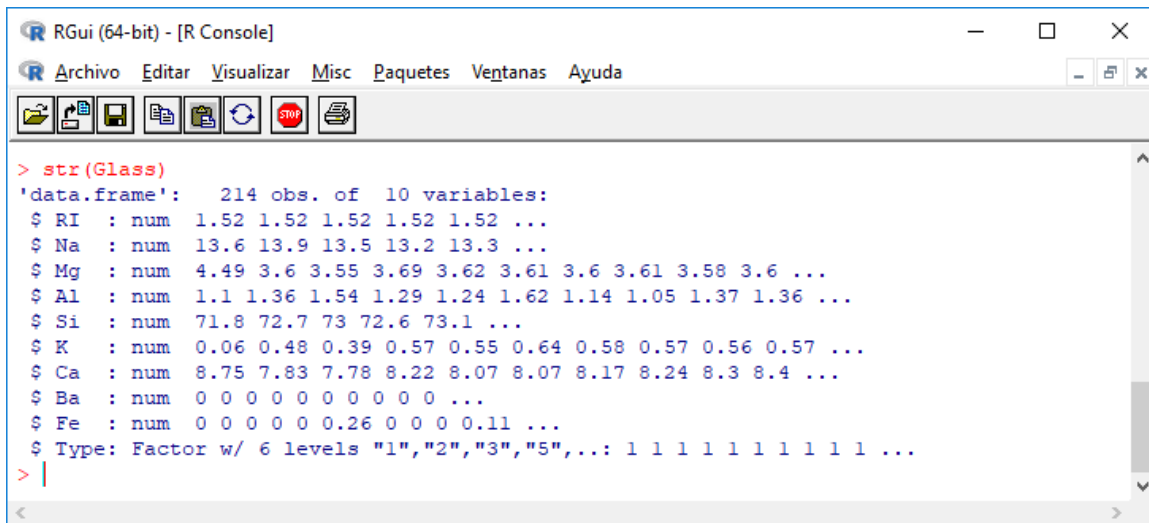
> library(e1071)
> library(rpart)
> data(Glass, package="mlbench")
> |

```

El paquete e1071 incluye clasificaciones y regresiones. Usaremos la función **svm()** para construir el modelo basado en Support Vector Machines.

Nuestro objetivo es predecir el tipo de vidrio en base a su análisis químico. Lo primero que debemos hacer es un análisis exploratorio de los datos, que dejamos de ejercicio para el lector.

La variable objetivo (la que queremos predecir) es 'Type', el resto lo podremos usar como predictores:

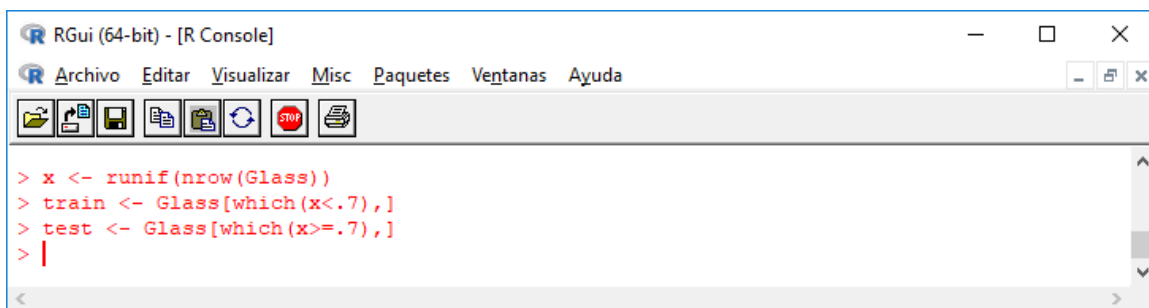


```

> str(Glass)
'data.frame': 214 obs. of 10 variables:
 $ RI : num 1.52 1.52 1.52 1.52 1.52 ...
 $ Na : num 13.6 13.9 13.5 13.2 13.3 ...
 $ Mg : num 4.49 3.6 3.55 3.69 3.62 3.61 3.6 3.61 3.58 3.6 ...
 $ Al : num 1.1 1.36 1.54 1.29 1.24 1.62 1.14 1.05 1.37 1.36 ...
 $ Si : num 71.8 72.7 73 72.6 73.1 ...
 $ K : num 0.06 0.48 0.39 0.57 0.55 0.64 0.58 0.57 0.56 0.57 ...
 $ Ca : num 8.75 7.83 7.78 8.22 8.07 8.07 8.17 8.24 8.3 8.4 ...
 $ Ba : num 0 0 0 0 0 0 0 0 0 0 ...
 $ Fe : num 0 0 0 0 0 0.26 0 0 0 0.11 ...
 $ Type: Factor w/ 6 levels "1","2","3","5",...: 1 1 1 1 1 1 1 1 1 1 ...
> |

```

Como siempre comenzamos separando los datos en dos conjuntos, uno para entrenamiento y otro para prueba:



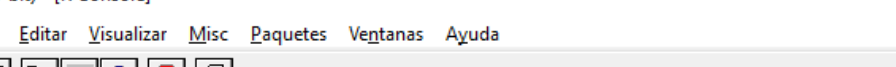
```

> x <- runif(nrow(Glass))
> train <- Glass[which(x<.7),]
> test <- Glass[which(x>=.7),]
> |

```

Como un ejercicio interesante (y como práctica!), vamos a comparar el rendimiento de un modelo basado en SVM usando la función `svm()` y otro basado en árboles de decisión usando `rpart()`.

Tanto para SVM como para el árbol de particionamiento usando **rpart()**, construiremos el modelo usando el set de entrenamiento (training), y predecimos los valores del set de validación (testing).



RGui (64-bit) - [R Console]

Archivo Editar Visualizar Misc Paquetes Ventanas Ayuda



```
> svm.model <- svm(Type~., data = train, cost = 100, gamma = 1)
> svm.pred <- predict(svm.model,test)
> tree.model <- rpart(Type~.,data = train)
> tree.pred <- predict(tree.model,newdata=test)
> |
```

El parámetro 'cost' de la función **svm()** es un parámetro de penalización general para la clasificación de clases, y 'gamma' es el parámetro que indica el kernel con las funciones de base radial.

Vamos a ver errores y aciertos de los modelos.

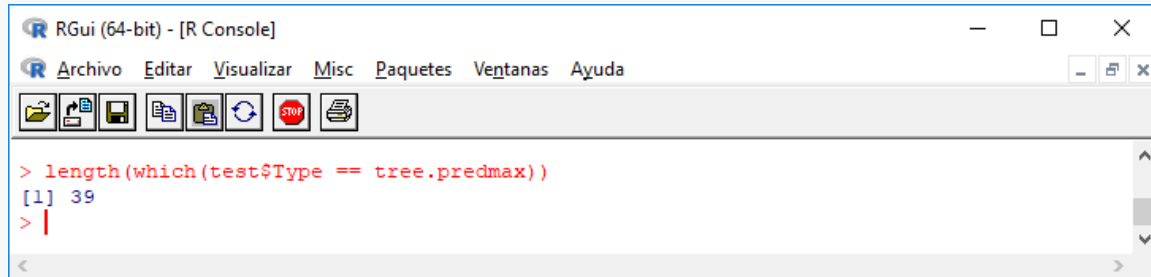
[illegible]

Fácilmente vemos que svm tiene 41 aciertos de los 68 casos. Se trata de un 60%

Para el árbol la cosa no es tan directa. Para cada uno de los puntos nos brindó la probabilidad de que pertenezcan a cada una de las 6 clases. Definimos un vector `tree.predmax` con tantas posiciones como `test`.

Luego le asignamos en cada posición el `which.max()` de cada fila de `tree.pred` que nos devuelve la posición en la que se encuentra el máximo valor de la fila.

Recién entonces estamos en condiciones de contar los aciertos:



```

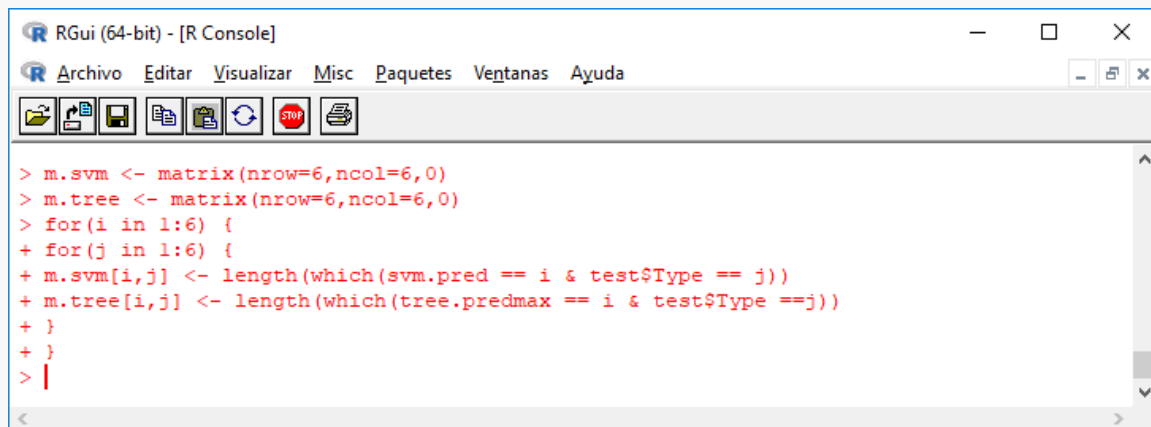
RGui (64-bit) - [R Console]
Archivo  Editar  Visualizar  Misc  Paquetes  Ventanas  Ayuda
[+] 39
> length(which(test$Type == tree.predmax))
[1] 39
>

```

En este caso particular, sin refinar ninguno de los dos `svm` le gana a `rpart` por poca diferencia.

Sin embargo esta es una forma bastante teórica de comparar los resultados. Los errores podrían no costar todos iguales.

Vamos a ver las matrices que miden la distribución de aciertos y errores en ambos métodos:



```

RGui (64-bit) - [R Console]
Archivo  Editar  Visualizar  Misc  Paquetes  Ventanas  Ayuda
> m.svm <- matrix(nrow=6,ncol=6,0)
> m.tree <- matrix(nrow=6,ncol=6,0)
> for(i in 1:6) {
+   for(j in 1:6) {
+     m.svm[i,j] <- length(which(svm.pred == i & test$Type == j))
+     m.tree[i,j] <- length(which(tree.predmax == i & test$Type ==j))
+   }
+ }
>

```

Y ahora veamos los resultados:

```

RGui (64-bit) - [R Console]
Archivo  Editar  Visualizar  Misc  Paquetes  Ventanas  Ayuda

[ ,1] [ ,2] [ ,3] [ ,4] [ ,5] [ ,6]
[1,]  19    6    2    0    0    0
[2,]   2   14    3    0    6    3
[3,]   0    1    1    0    0    0
[4,]   0    0    0    0    0    0
[5,]   0    0    0    0    0    0
[6,]   0    0    0    0    0    1
> m.tree
[ ,1] [ ,2] [ ,3] [ ,4] [ ,5] [ ,6]
[1,]  19    1    4    0    0    0
[2,]   2   20    2    0    0    1
[3,]   0    0    0    0    0    0
[4,]   0    0    0    0    6    3
[5,]   0    0    0    0    0    0
[6,]   0    0    0    0    0    0
> |

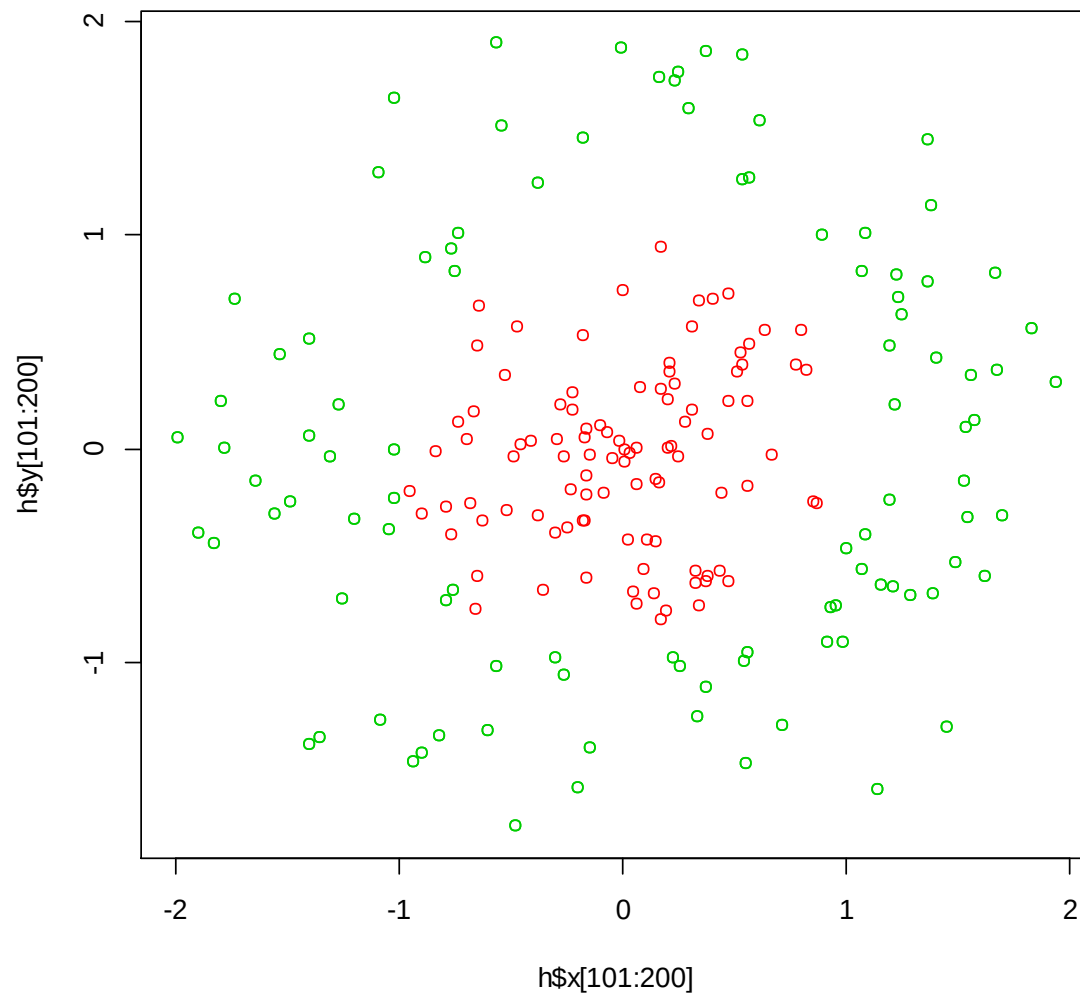
```

De las tablas de comparación vemos que el modelo basado en SVM resulta mejor que el basado en **rpart()**, pues el número de valores no nulos fuera de la diagonal es menor.

Analicemos ahora una aplicación de regresión no lineal. Realicemos el mismo procedimiento que antes para separar los datos en un set de entrenamiento y otro de validación.

Linealización:

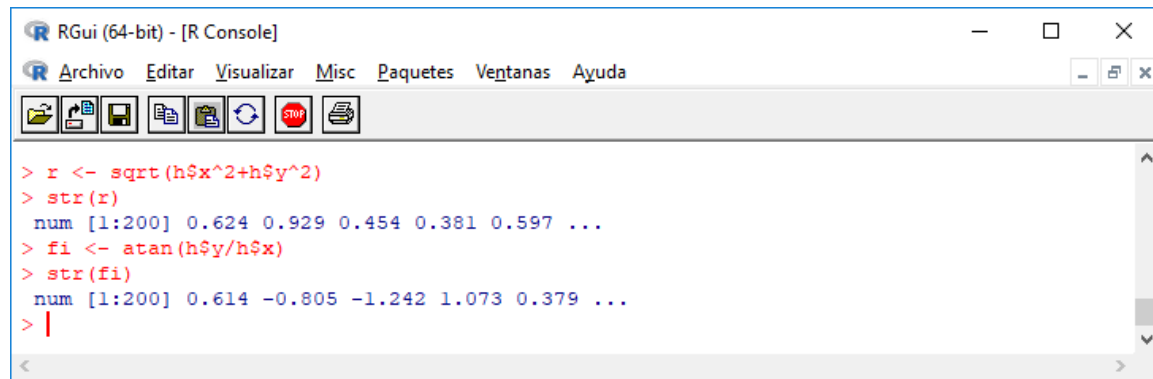
No siempre los puntos son separables por una línea recta. ¿Qué pasa si nos encontramos con esta distribución?



Nuestro problema es que cualquier recta que podamos tirar dejará los puntos mal repartidos.

Qué pasaríamos si en vez de una recta tiráramos un círculo con centro en 0,0 y radio 1? Pues parecería que ese círculo dejaría muchos puntos rojos dentro y muchos puntos verdes fuera.

Entonces se nos ocurre plantear una transformación de coordenadas de cartesianas a polares:



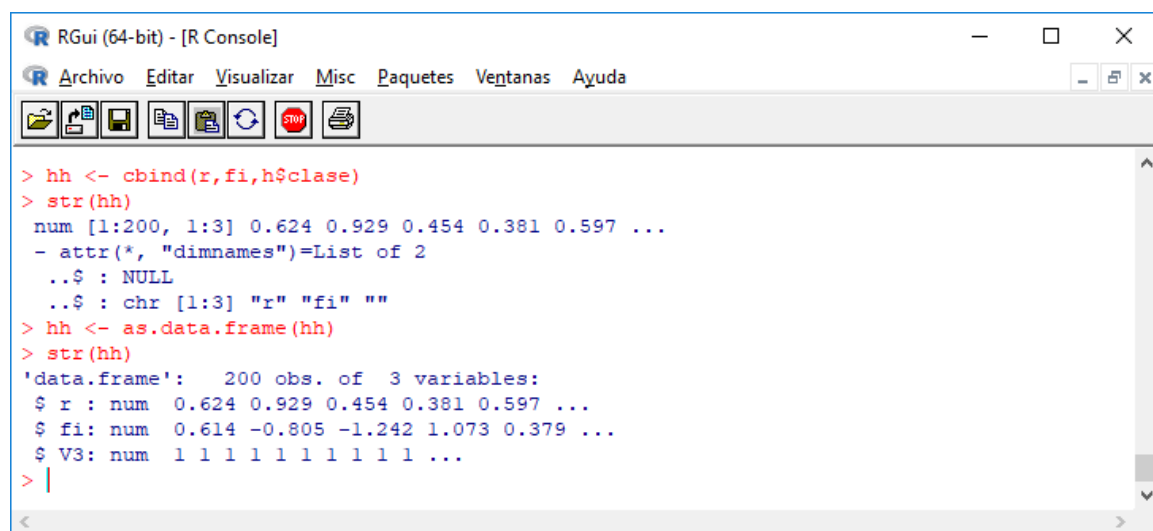
```

RGui (64-bit) - [R Console]
Archivo  Editar  Visualizar  Misc  Paquetes  Ventanas  Ayuda

> r <- sqrt(h$x^2+h$y^2)
> str(r)
  num [1:200] 0.624 0.929 0.454 0.381 0.597 ...
> fi <- atan(h$y/h$x)
> str(fi)
  num [1:200] 0.614 -0.805 -1.242 1.073 0.379 ...
> |

```

Y luego combinamos en un dataframe:



```

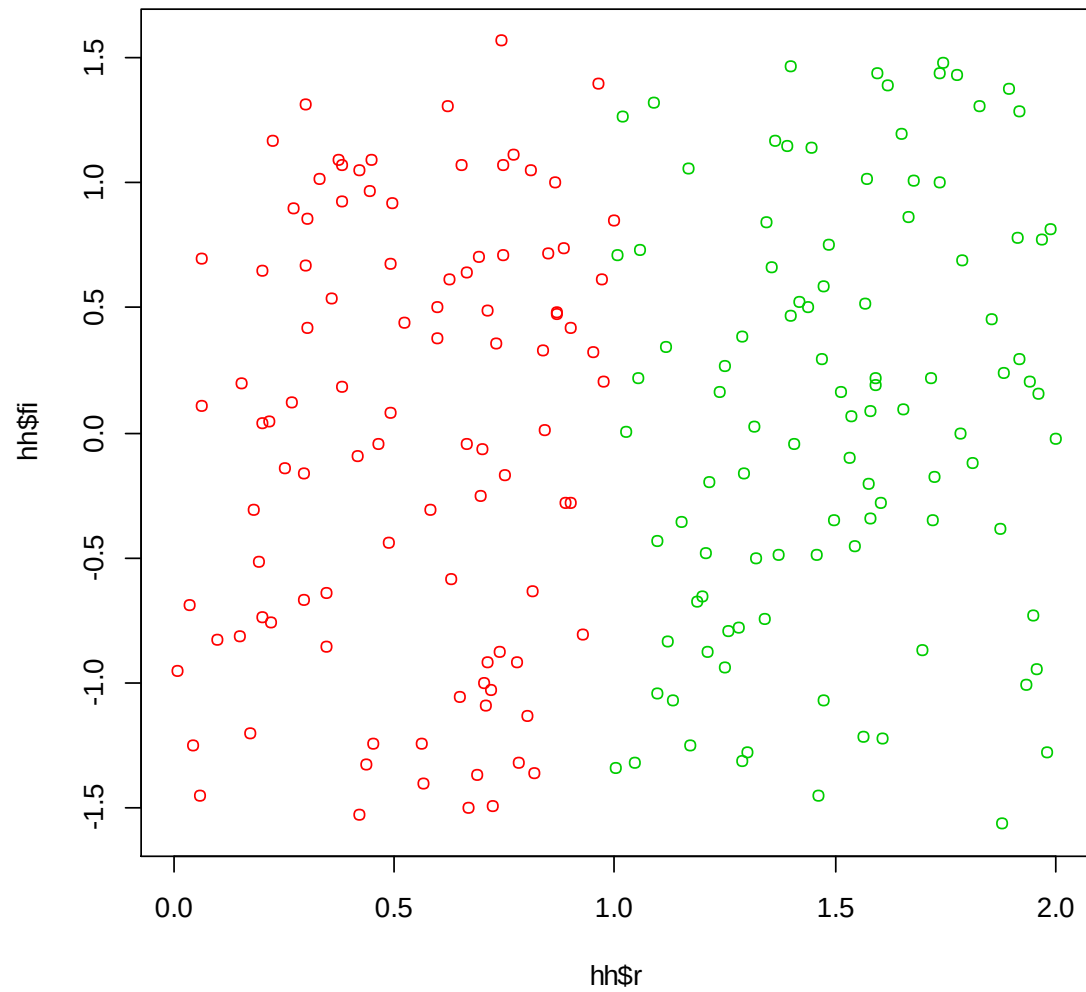
RGui (64-bit) - [R Console]
Archivo  Editar  Visualizar  Misc  Paquetes  Ventanas  Ayuda

> hh <- cbind(r,fi,h$class)
> str(hh)
  num [1:200, 1:3] 0.624 0.929 0.454 0.381 0.597 ...
- attr(*, "dimnames")=List of 2
 ..$ : NULL
 ..$ : chr [1:3] "r" "fi" ""
> hh <- as.data.frame(hh)
> str(hh)
'data.frame':   200 obs. of  3 variables:
 $ r : num  0.624 0.929 0.454 0.381 0.597 ...
 $ fi: num  0.614 -0.805 -1.242 1.073 0.379 ...
 $ V3: num  1 1 1 1 1 1 1 1 1 1 ...
> |

```

Notamos que la variable clase ahora se llama V3 como resultado la conversión

Veamos como lucen los datos ahora:



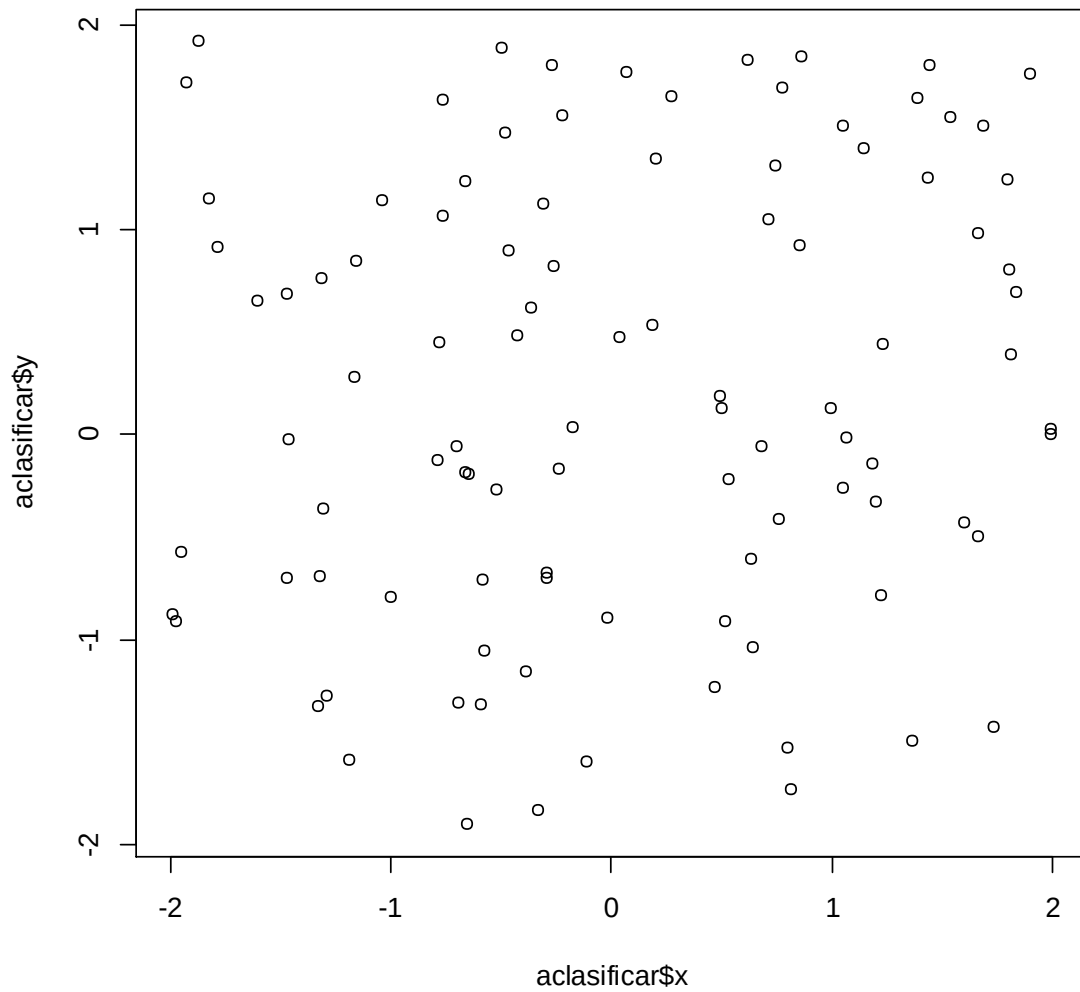
Ahora no tenemos problemas con el trazado de la recta...

Construimos el modelo:

```
RGui (64-bit) - [R Console]
Archivo Editar Visualizar Misc Paquetes Ventanas Ayuda
> svm.model <- svm(V3 ~ ., data = hh)
> |
```

Si tenemos un conjunto de puntos por clasificar como por ejemplo:

```
RGui (64-bit) - [R Console]
Archivo Editar Visualizar Misc Paquetes Ventanas Ayuda
> plot(aclasificar$x,aclasificar$y)
> |
```

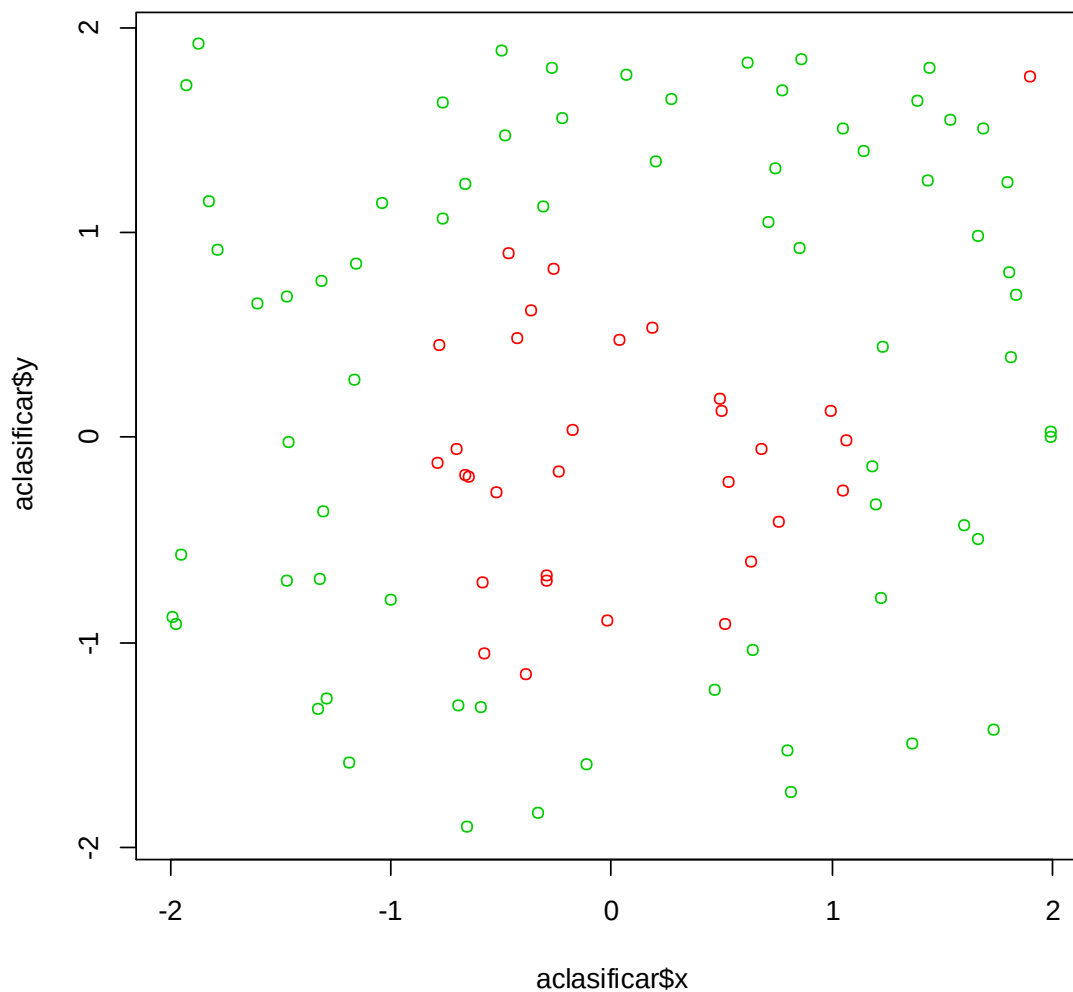


Lo primero que tenemos que hacer es aplicarle la transformación al dataframe `aclasificar`

Vamos a utilizar `predict` sobre estos puntos aplicando el modelo `svm` que hemos entrenado:

```
RGui (64-bit) - [R Console]
Archivo  Editar  Visualizar  Misc  Paquetes  Ventanas  Ayuda
[Icons]
> clasificacion <- predict(svm.model,newdata= transformado,type='class')
> hist(clasificacion)
> plot(aclasificar$x,aclasificar$y, col = 2+(clasificacion-.8))
> |
```

Y nos da:



Hemos conseguido utilizar la clasificación que generamos con los datos transformados sobre el conjunto original sin transformar.